Chapter 06: Microservices Architecture

Software Engineering

Jeremy Evert

SWOSU

April 14, 2026

Chapter Overview

1. Introduction
2. Microservices
3. Microservices Architecture
4. RESTful Services
5. Service Deployment
6. Microservices in Practice

Learning Objectives

- Define a software service and microservices architecture.
- Explain why microservices became important in modern software systems.
- Identify the essential characteristics of microservices.
- Describe the importance of low coupling and high cohesion.
- Explain practical guidelines for deciding the scope of a microservice.

Why This Chapter Matters

- Modern software systems are often too large to build and deploy as one giant block.
- Microservices offer a way to split a system into independent parts.
- This can make change, scaling, and deployment easier.
- But it also creates new challenges in communication, coordination, data consistency, and failure handling.

Why System Decomposition Matters

- One of the most important software architecture decisions is how a system is decomposed into components.
- Good decomposition supports:
  - parallel development by different teams,
  - easier replacement or reuse of components,
  - scalability and elasticity in cloud environments,
  - improved reliability through isolation.
- In distributed cloud systems, components must be easy to replicate, move, and update.

What Is a Software Service?

- A software service is a component that can be accessed remotely over the Internet.
- A request is sent to the service, and the service returns a result.
- The service exposes a published interface.
- Internal implementation details are hidden from the service user.
- The manager of a service is called the **service provider**.

Why Services Fit Cloud-Based Software

- Services are especially useful in cloud-based systems because they can be:
  - distributed across multiple computers,
  - replicated to handle higher loads,
  - updated independently,
  - moved from one server to another.
- Services typically avoid maintaining local internal state.
- Persistent information is usually stored in a database or maintained by the service requester.

From Service-Oriented Architecture to Microservices

- The idea of service-oriented architecture is not new.
- Earlier service systems often relied on heavyweight web-service standards such as:
  - SOAP
  - WSDL
  - XML-based messaging
- These approaches were often complex and created performance overhead.
- Modern microservices favor simpler interfaces and lighter communication mechanisms.

What Are Microservices?

- Microservices are small, independent services that can be combined to create applications.
- Each microservice should focus on a single business capability or closely related responsibility.
- A microservice should be modifiable and redeployable without forcing changes to the rest of the system.
- Microservices exchange messages to request services and return responses.

Essential Characteristics of Microservices

- **Self-contained:** a service manages its own data and user-facing behavior.
- **Lightweight:** services communicate with low-overhead protocols.
- **Implementation independent:** different services may use different languages or technologies.
- **Independently deployable:** each service can run and be updated on its own.
- **Business-oriented:** services are organized around business needs, not just technical utilities.

Core Design Goal: Low Coupling

- **Coupling** measures how much one component depends on other components.
- Low coupling means a service has few relationships with other services.
- This is important because:
  - services can change more independently,
  - updates are less likely to break the rest of the system,
  - systems are easier to scale and maintain.

Core Design Goal: High Cohesion

- **Cohesion** measures how closely related the responsibilities inside a component are.
- High cohesion means the parts of a service all contribute to one focused purpose.
- High cohesion is desirable because:
  - the service is easier to understand,
  - the service is easier to test,
  - fewer cross-service calls are needed during execution.

Single Responsibility in Practice

- Microservice design is strongly influenced by the single responsibility principle.
- Each service should do one thing well.
- However, "one thing" should be interpreted carefully.
- Responsibilities should be grouped in ways that reduce unnecessary shared databases and excessive service-to-service communication.
- The goal is not the smallest possible service.
- The goal is the **right boundary** for an independently useful service.

How Big Should a Microservice Be?

- There is no perfect size rule for microservices.
- A practical guideline is the **rule of twos**:
  - a microservice should be developable, testable, and deployable in about two weeks or less,
  - the development team should be small, often no larger than a "two-pizza team."
- Size should be judged by responsibility, independence, and manageability, not by lines of code.

Key Takeaways from the Introduction

- Microservices grew out of service-based software ideas but use simpler, lighter communication styles.
- They are designed to support scalability, flexibility, and independent deployment.
- Good microservice design aims for:
  - low coupling,
  - high cohesion,
  - strong business alignment,
  - practical service boundaries.

Microservices Communicate by Message Exchange

- Microservices interact by sending requests and receiving responses.
- Messages may include:
  - administrative information,
  - a service request,
  - the data needed to deliver that service.
- The response includes the result of the requested service.
- This communication model is central to distributed systems design.

Looking Ahead

Next: Section 6.1 Microservices

We will now look more closely at the detailed characteristics, design tradeoffs, and examples of microservices.

Section 6.1: Microservices

- Microservices are small, independent services that can be combined to create applications.
- A service should be changeable and redeployable without forcing changes to other services.
- Microservices communicate through message exchange.
- Good microservice design aims for:
  - low coupling,
  - high cohesion,
  - clear business responsibility.

Essential Characteristics of Microservices

- **Self-contained:** the service manages its own data and user-facing behavior.
- **Lightweight:** communication overhead should be low.
- **Implementation independent:** services may use different languages or technologies.
- **Independently deployable:** each service can run and be updated on its own.
- **Business-oriented:** services should implement business capabilities, not just technical fragments.

Microservices Communicate by Exchanging Messages

- Services interact by sending requests and receiving responses.
- A message may include:
  - administrative information,
  - the requested service,
  - the data needed to perform that service.
- The reply contains the result of the request.
- Example:
  - an authentication service may send login information,
  - a login service may return a token or an error.

Core Design Goal: Low Coupling

- **Coupling** measures how much one component depends on other components.
- Low coupling means a service has relatively few relationships with other services.
- This matters because:
  - services can be changed independently,
  - updates are less likely to ripple through the system,
  - maintenance and scaling become easier.
- As long as the interface stays stable, a loosely coupled service can evolve without forcing change elsewhere.

Core Design Goal: High Cohesion

- **Cohesion** measures how closely related the responsibilities inside a component are.
- High cohesion means all parts of the component contribute to one focused purpose.
- This matters because:
  - the service is easier to understand,
  - the service is easier to test,
  - fewer cross-service calls are needed during execution.
- High cohesion reduces communication overhead and keeps service boundaries meaningful.

Single Responsibility, with a Warning Label

- Microservice design is influenced by the **single responsibility principle**.
- Each service should do one thing well.
- However, "one thing" should not be interpreted too narrowly.
- If responsibilities are split too aggressively, services may:
  - depend on shared databases,
  - communicate too often,
  - become more tightly coupled.
- The goal is not the smallest service.
- The goal is the **right service boundary**.

How Big Should a Microservice Be?

- There is no perfect formula for microservice size.
- Lines of code are not a useful measure.
- A practical guideline is the **rule of twos**:
  - a service should be developable, testable, and deployable in about two weeks or less,
  - the team should be small, often no larger than a "two-pizza team."
- Microservice size should be judged by independence, manageability, and responsibility.

Microservices Are Not Tiny Functions

- A microservice may have a single responsibility, but that does not mean it is just one short function.
- A real service often includes:
  - service logic,
  - data management,
  - security code,
  - interface code,
  - testing and deployment support.
- Teams may need significant effort to make a service fully independent and production-ready.

Example: Password Management Microservice

- A password-management service may include user-facing functions such as:
  - create password,
  - change password,
  - check password,
  - recover password.
- It may also include supporting functions such as:
  - password validity checks,
  - backup and recovery,
  - database integrity checks,
  - repair operations.

Support Code Inside a Microservice

- To remain independent, a microservice needs more than core business logic.
- Common supporting responsibilities include:
  - message management,
  - failure management,
  - user interface implementation,
  - data consistency management.
- In many systems, the supporting code can be as important as the main service functionality.

Why Testing and Operations Matter

- Service teams are usually responsible for more than initial coding.
- They may also handle:
  - automated testing,
  - interaction testing with other services,
  - deployment support,
  - ongoing maintenance after release.
- Independent deployment only works well when testing and operational support are built in.

Section 6.1 Recap

- Microservices are independent, business-oriented services that collaborate through message exchange.
- Strong microservice design aims for low coupling and high cohesion.
- Microservices should be small enough to manage, but large enough to own a complete responsibility.
- A production microservice includes both service functionality and substantial support code.

Microservice Support Code at a Glance

Microservice X	
Service Functionality	
Message management	Failure management
UI implementation	Data consistency management

Looking Ahead

Next: Section 6.2 Microservices Architecture

We now move from the characteristics of individual microservices to the broader architectural style built from those services.

Section 6.2: Microservices Architecture

- A microservices architecture is an **architectural style**, not a fixed layered template.
- It is commonly used for web-based client-server systems.
- The server side is implemented as a set of interacting microservices.
- This style is intended to address key problems in distributed systems design.

Microservices Architecture as a Style

- A layered architecture defines a common set of components used across similar applications.
- A microservices architecture does something different.
- It provides a tried-and-tested **way of organizing** distributed software.
- In web systems, the client interacts with an application whose server-side behavior is split across multiple services.

Why Use Microservices Architecture?

- It can improve scalability.
- It can improve resilience.
- It can support independent deployment of services.
- It can allow services to be restarted or updated without bringing down the whole system.
- In cloud environments, scaling out by replicating services is often cheaper than scaling up a single large server.

Example Architecture: Client, Gateway, and Services

- A common design uses:
  - a client application,
  - an API gateway,
  - a collection of backend services.
- The gateway routes and translates requests from the client into calls to the appropriate microservices.
- This lets internal services change without forcing immediate changes to the client app.

A Photo-Printing Example

- The book uses a photo-printing system as an example.
- Possible services include:
  - registration,
  - authentication,
  - upload,
  - payment,
  - printing,
  - dispatch.
- An API gateway sits between the mobile app and these backend services.

Architectural Design Decisions

- In a microservices system, the development teams for services are autonomous.
- However, the overall system still has to work as a coordinated whole.
- Key architectural questions include:
  - How should the system be decomposed?
  - How should services communicate?
  - How should data be distributed and shared?
  - How should the system be coordinated?
  - How should failures be detected and managed?

How Should the System Be Decomposed?

- Too many microservices increase communication overhead and complexity.
- Too few microservices make services less autonomous and harder to change independently.
- Decomposition should balance:
  - performance,
  - independence,
  - maintainability,
  - business capability alignment.

Guidelines for Decomposition

1. Balance fine-grain functionality and performance.
2. Follow the common closure principle.
3. Associate services with business capabilities.
4. Design services to own the data they need.

Common Closure and Business Capability

- The **common closure principle** says that elements likely to change together should be located together.
- This reduces the chance that a single requirement change forces changes across several services.
- Services should also be aligned with **business capabilities**, not arbitrary technical fragments.
- A useful service boundary usually reflects a coherent real-world responsibility.

Starting from a Monolith

- A practical way to identify microservices is to begin with an existing monolithic architecture.
- Then identify parts of the system that encapsulate distinct responsibilities.
- These responsibilities can be extracted into services over time.
- In other words, refactoring a monolith can be a realistic path to microservices.

Service Communications

- Services communicate by exchanging messages.
- Important design choices include:
  - synchronous or asynchronous interaction,
  - direct service-to-service communication or message broker middleware,
  - the protocol and message structure to use.
- These choices affect performance, flexibility, and complexity.

Synchronous vs. Asynchronous Communication

- **Synchronous communication:**
  - a service waits for a reply before continuing,
  - simpler to understand,
  - easier to develop initially.
- **Asynchronous communication:**
  - services do not have to wait for immediate responses,
  - can improve resilience and decoupling,
  - often requires more infrastructure and coordination.

Direct Communication vs. Message Brokers

- With **direct communication**, one service calls another service directly.
- This is often easier to understand and faster to implement.
- With a **message broker**, services communicate through middleware.
- Brokers can support buffering, decoupling, and asynchronous behavior.
- However, brokers add infrastructure complexity.

Communication Protocols

- A message protocol defines:
  - how messages are structured,
  - what information is included,
  - how senders and receivers interpret the message.
- Modern systems often use:
  - HTTP
  - JSON
  - REST-style interfaces
- The protocol should be standard enough that all services can interoperate consistently.

Data Distribution and Sharing

- In principle, each microservice should manage its own data.
- In reality, complete data independence is difficult.
- Different services often need related or overlapping data.
- This creates one of the hardest problems in microservices:
  - how to share data while preserving autonomy and consistency.

Data Design Guidelines

1. Isolate data within each service as much as possible.
2. If sharing is unavoidable, make shared data read-only when possible.
3. If services are replicated, include a mechanism to keep replicas consistent.

ACID Consistency vs. Microservice Independence

- Traditional multi-tier systems often use a shared database with ACID transactions.
- ACID transactions ensure updates are completed as a single consistent unit.
- This works well for strong consistency.
- However, shared databases reduce microservice independence.
- If strict consistency is required, you may need to accept tighter coupling.

Eventual Consistency

- In many microservices systems, replicas may not be immediately identical.
- Instead, the system aims for **eventual consistency**.
- Updates are recorded, propagated, and applied across replicas over time.
- This improves scalability and service autonomy,
- but it means the system may be temporarily inconsistent.

Dealing with Inconsistency

- Two important data problems are:
  - **dependent data inconsistency** caused by actions or failures in one service,
  - **replica inconsistency** when replicated databases temporarily disagree.
- A common technique is to use a **transaction log** of pending updates.
- Compensating actions may also be needed when part of a multi-service workflow fails.

Service Coordination

- Many business operations involve several services working together in a workflow.
- Example:
  - user login,
  - password checks,
  - lockout rules,
  - authentication success or failure.
- The architectural question becomes:
  - who controls the workflow?

Orchestration

- In **orchestration**, a central controller manages the workflow.
- The controller tells each service when to act.
- Advantages:
  - easier to understand the overall process,
  - easier to debug,
  - easier to identify failures.
- Disadvantage:
  - adds a central coordinating component.

Choreography

- In **choreography**, there is no central controller.
- Services observe events and decide for themselves when to act next.
- This can reduce central control and loosen coupling.
- However:
  - workflows are harder to understand,
  - debugging is harder,
  - recovery from failure can be harder.

Failure Management

- Failures are normal in distributed systems.
- Even if individual services are reliable, failures become inevitable at scale.
- The architecture must be designed to:
  - detect failures,
  - report failures,
  - recover from failures,
  - degrade gracefully when recovery is not immediate.

Common Failure Types

- **Internal service failures:**
  - bugs,
  - invalid data handling,
  - broken internal links or logic.
- **External service failures:**
  - dependency unavailable,
  - dependency too slow,
  - dependency returns an error.
- **Performance failures:**
  - degraded responsiveness under load,
  - overload conditions,
  - timeout problems.

Failure Responses

- A simple approach is to use standard status codes and agreed-upon error responses.
- Systems may use:
  - timeouts,
  - retries,
  - fallback behaviors,
  - logging and monitoring.
- The architecture should make failures visible and diagnosable.

Circuit Breakers

- A **circuit breaker** prevents repeated calls to a failing service.
- If a service repeatedly times out or fails:
  - the breaker marks it as unavailable,
  - later requests are blocked or rerouted,
  - the system periodically tests whether the service has recovered.
- This prevents one struggling service from dragging the rest of the system into the swamp.

Monitoring and Observability

- In practice, microservices need strong monitoring support.
- Useful observations include:
  - service response times,
  - bottlenecks,
  - frequency of failures,
  - patterns of degraded performance.
- Logs and dashboards help system managers understand where improvements are needed.

Section 6.2 Recap

- Microservices architecture is a distributed architectural style built from interacting services.
- Architects must make decisions about decomposition, communication, data sharing, coordination, and failure handling.
- These decisions involve tradeoffs between simplicity, autonomy, consistency, and resilience.
- A good architecture is not just a set of services.
- It is a set of services that can work together without collapsing into distributed chaos.

Looking Ahead

Next: Section 6.3 RESTful Services

We next examine REST as a common communication style for service interaction.

Section 6.3: RESTful Services

- Microservices may communicate synchronously or asynchronously using different protocols.
- One of the most widely used approaches for service interaction is **REST**.
- RESTful services follow a set of conventions built on top of HTTP.
- In practice, REST is one of the most common ways for microservices to exchange information.

What REST Really Is

- Strictly speaking, REST is not a single formal protocol.
- It is an **architectural style** for service communication.
- RESTful services:
  - use HTTP,
  - organize information as resources,
  - exchange representations of those resources.
- REST stands for **Representational State Transfer**.

Core Idea Behind REST

- REST is based on transferring representations of digital resources between server and client.
- A resource could be:
  - a book,
  - a customer record,
  - a traffic incident,
  - an uploaded image,
  - an order.
- The same resource may have different representations, such as JSON, XML, or HTML.

RESTful Service Principles

- **Use HTTP verbs** to express operations.
- **Be stateless:** the server should not rely on remembered client session state.
- **Use URI-addressable resources:** each resource should have a unique identifier.
- **Use structured representations:** typically JSON, and sometimes XML.

Why Statelessness Matters

- In a stateless service, each request contains the information needed to process it.
- The server does not depend on stored interaction state from earlier requests.
- This supports:
  - scalability,
  - easier replication,
  - simpler failure recovery.
- Statelessness fits well with cloud-based microservices.

Resources and URIs

- In REST, a resource is accessed through a **URI**.
- URIs should reflect a meaningful hierarchical structure.
- Example idea:
  - road incidents organized by road, place, direction, and incident number
- A good URI structure makes resources easier to identify and operate on.

CRUD Operations in REST

- RESTful services commonly support four fundamental operations:
  - **Create**
  - **Read**
  - **Update**
  - **Delete**
- These are often mapped onto standard HTTP verbs.

HTTP Verbs and REST Operations

Operation	Typical HTTP Verb
Create	POST
Read	GET
Update	PUT
Delete	DELETE

- This mapping gives RESTful services a consistent structure.

Example: Traffic Incident Service

- Imagine a service that stores traffic incidents.
- Example operations might include:
  - **GET** to retrieve incident information,
  - **POST** to add a new incident,
  - **PUT** to update an incident,
  - **DELETE** to remove an incident.
- This is a good example of a resource-oriented REST design.

Request and Response Processing

- A RESTful microservice accepts an HTTP request,
- processes the request,
- performs the required service actions,
- and generates an HTTP response.
- This creates a predictable request/response flow.

HTTP Request and Response Structure

- A typical HTTP request includes:
  - an HTTP verb,
  - a URI,
  - an HTTP version,
  - request headers,
  - optionally, a request body.
- A typical HTTP response includes:
  - an HTTP version,
  - a response code,
  - response headers,
  - optionally, a response body.

Important HTTP Header Information

- **Accept** specifies what response content types the client can handle.
- **Content-Type** specifies the type of the request or response body.
- **Content-Length** specifies the size of the message body.
- These headers help clients and services interpret exchanged data correctly.

JSON vs. XML

- RESTful services often return structured data in **JSON**.
- XML can also be used, but JSON is usually preferred in modern systems.
- JSON is popular because it is:
  - simpler,
  - lighter weight,
  - easier to parse in many environments.
- XML is more flexible, but often more verbose.

Example Resource Representation

- A traffic incident resource might include:
  - incident ID,
  - date,
  - time,
  - road ID,
  - place,
  - direction,
  - severity,
  - description.
- REST returns a representation of that resource, not just a vague blob of text.

Why REST Works Well for Microservices

- REST is widely understood and widely supported.
- It fits naturally with web technologies and HTTP infrastructure.
- It encourages:
  - simple interfaces,
  - stateless interaction,
  - consistent operations on resources.
- This makes it a practical communication style for many microservice systems.

Limits of REST

- REST is useful, but it is not always the only or best choice.
- Some systems may need:
  - asynchronous messaging,
  - event-based communication,
  - specialized protocols.
- REST is best seen as one important tool in the microservices toolbox.

Section 6.3 Recap

- RESTful services use HTTP and treat information as resources.
- Resources are identified by URIs and manipulated using standard HTTP verbs.
- REST emphasizes stateless interaction and structured representations.
- JSON is commonly preferred for modern microservice communication.

Looking Ahead

Next: Section 6.4 Service Deployment

We next examine what happens after development, when services must actually be deployed, updated, and operated.

Section 6.4: Service Deployment

- After a system is built, it must be deployed, monitored, and updated.
- In microservices systems, deployment is more complex than in monolithic systems.
- Different services may use different languages, libraries, databases, and runtime environments.
- This makes deployment a major architectural and operational concern.

Why Deployment Is Harder with Microservices

- A microservices system may contain tens or hundreds of services.
- Each service may have:
  - its own dependencies,
  - its own runtime requirements,
  - its own update cycle.
- There is no single standard deployment configuration for all services.
- A separate operations team can easily become a deployment bottleneck.

From Separate Ops to DevOps

- Traditionally, operations and development were separate activities.
- In microservices systems, it is common for development teams to take responsibility for:
  - deployment,
  - service management,
  - ongoing operational support.
- This combined approach is commonly called **DevOps**.

Continuous Deployment

- A general principle of microservice-based development is that teams own deployment of their service.
- A common practice is **continuous deployment**.
- When a change is made and validated, the modified service is re-deployed automatically.
- This contrasts with older release models where changes were bundled into occasional large releases.

A Continuous Deployment Pipeline

- A typical pipeline includes:
  - commit code changes,
  - run unit tests,
  - build the test system,
  - run integration tests,
  - package the service,
  - deploy the new version,
  - run acceptance or whole-system tests,
  - replace the current service if all tests pass.
- If a step fails, the change is rejected.

Why Automation Matters

- Continuous deployment depends on automation.
- Automated tests must check that:
  - the modified service works correctly,
  - it still interacts correctly with other services,
  - the whole system remains stable after deployment.
- Without automation, rapid deployment quickly becomes dangerous chaos.

Containers as Deployment Units

- Containers are often the preferred way to package microservices for deployment.
- A container includes the executable service and the software environment it needs.
- This makes deployment more portable across servers.
- Containers reduce the need for service teams to worry about low-level server configuration differences.

Why Containers Help

- Containers allow dependencies to be defined in advance.
- A service can often be deployed by:
  - building the container image,
  - loading it onto a server or cloud platform,
  - starting the containerized service.
- This makes deployment more repeatable and easier to automate.

Managing Many Containers

- Large microservice systems may involve tens, hundreds, or thousands of containers.
- Managing them manually is difficult.
- Container-management systems help automate:
  - deployment,
  - scheduling,
  - service discovery,
  - load balancing,
  - resource management.

Kubernetes and Container Orchestration

- The book highlights **Kubernetes** as an example of a container-management platform.
- Kubernetes helps manage container deployment across clusters of servers.
- It supports:
  - automated deployment,
  - scheduling,
  - service discovery,
  - load balancing.
- This kind of orchestration becomes essential at scale.

Deployment Risk

- A major deployment risk is that a new service version may interact badly with existing services.
- Even when tests pass, unexpected behavior may still appear in production.
- Testing reduces risk, but it cannot eliminate it completely.
- Therefore, deployed services must be monitored carefully after release.

Monitoring New Versions

- After deployment, the system should watch for:
  - failures,
  - degraded performance,
  - unexpected interactions,
  - incorrect responses.
- Monitoring helps detect whether a new version is safe to keep in production.
- If serious problems appear, the system should be able to roll back.

Rollback and Version Switching

- One practical strategy is to keep the old version available while introducing the new version.
- If the monitoring system detects problems, traffic can be switched back to the older version.
- This makes rollback faster and safer than rebuilding the old version from scratch.
- API gateways can help route requests to the currently active service version.

Versioned Services

- In some systems, service versions are identified explicitly.
- For example, an API change may require clients or gateways to distinguish between:
  - version 001,
  - version 002,
  - and later versions.
- Over time, clients and gateways can be updated to use the newer service version.

Big Picture: What Deployment Requires

- Effective microservice deployment depends on:
  - team ownership,
  - automation,
  - testing,
  - containers,
  - monitoring,
  - safe rollback mechanisms.
- Deployment is not an afterthought.
- It is part of the architecture.

Section 6.4 Recap

- Microservice deployment is more complex than monolithic deployment because services are independently developed and updated.
- DevOps and continuous deployment help teams manage this complexity.
- Containers provide a portable deployment unit, and Kubernetes helps manage containers at scale.
- Monitoring and rollback are essential because even tested deployments can fail in production.

Looking Ahead

Next: Summary and Further Reading

We now move from chapter content into final takeaways and recommended reading.

Microservices in Practice

- Microservices are not just a programming pattern.
- They are a way of organizing software, teams, deployment, and system evolution.
- Their goal is to make large systems easier to change, scale, and maintain.
- But they do this by introducing new architectural and operational complexity.

The Big Idea

- A microservices system is built from independent services.
- Each service owns a focused responsibility.
- Each service communicates with others through lightweight interfaces.
- The hope is that independent parts can evolve without breaking the whole system.

Why Teams Use Microservices

- To scale parts of a system independently
- To let teams work more autonomously
- To deploy and update services separately
- To improve resilience by isolating failures
- To support modern cloud-based software systems

What Microservices Demand in Return

- More network communication
- More coordination between services
- Harder data consistency problems
- More deployment and monitoring complexity
- More responsibility placed on development teams

The Core Design Tension

- Microservices promise independence.
- Real systems still need communication, shared workflows, and consistent data.
- So the architect's job is always a balancing act:
  - independence vs. coordination
  - flexibility vs. complexity
  - speed vs. consistency
  - autonomy vs. control

What Good Microservice Design Tries to Achieve

- Low coupling between services
- High cohesion within each service
- Clear business-oriented boundaries
- Lightweight communication
- Independent deployment
- Sensible ownership of data and responsibility

A Practical Warning

- Microservices are not automatically better than monoliths.
- A badly designed microservices system can be harder to understand, test, deploy, and debug than a monolithic system.
- Splitting a system into services is only helpful when the service boundaries are meaningful.
- Small pieces are useful only when they reduce pain rather than multiply it.

When Microservices Make Sense

- The system is large and likely to evolve
- Different parts of the system change at different rates
- Different teams need to work independently
- Some services need to scale much more than others
- The organization has the tooling and discipline to handle distributed systems

When Microservices May Be the Wrong Choice

- The system is still small or simple
- The team is small and highly centralized
- The deployment environment is immature
- Strong consistency is required everywhere
- The cost of coordination is greater than the value of independence

What Students Should Remember Most

1. A microservice is a focused, independently deployable service.
2. Communication, coordination, and data management become central design problems.
3. REST is one common communication style, but not the only one.
4. Deployment, monitoring, rollback, and DevOps are part of the architecture.
5. Good microservices are not just small. They are well-bounded.

The Practical Takeaway

- Microservices are a powerful architectural style for building large, evolving systems.
- They work best when the system is carefully decomposed and supported by strong engineering practices.
- In the real world, success depends just as much on testing, deployment, monitoring, and team discipline as on code design.
- Architecture is not just about structure.
- It is about making change survivable.

Discussion Questions

1. Why might a company choose microservices instead of a monolithic architecture?
2. What new problems appear when a system is split into many services?
3. Why are low coupling and high cohesion so important in microservices?
4. When might a monolithic system actually be the better engineering decision?

Further Reading

- Sommerville, *Software Engineering*
- Sam Newman, *Building Microservices*
- Martin Fowler on Microservices

Questions?